

Déploiement d'un modèle de langage (LLM) avec Docker et Kubernetes

Yacine Mbaye
École Polytechnique de Thiès
DIC2 - Génie Informatique et Télécommunications

Septembre 2026

1 Introduction

Ce projet consiste à déployer une pile applicative composée d'Ollama, moteur d'exécution de modèles de langage en local, et d'Open WebUI, interface web de discussion placée par-dessus. Le déploiement se fait en deux temps : d'abord avec Docker Compose, comme socle de développement rapide, puis par une migration vers Kubernetes respectant les bonnes pratiques d'orchestration (persistance des données, santé des services, configuration externalisée, exposition réseau).

Trois modèles ont été rendus disponibles : Llama 3.2 (3B), Mistral (7B) et Gemma 3 (4B).

2 Socle Docker Compose

L'architecture de départ repose sur deux services définis dans `docker-compose.yaml`.

Le service `ollama` expose l'API d'inférence sur le port 11434, avec un volume nommé qui évite de re-télécharger les modèles à chaque redémarrage.

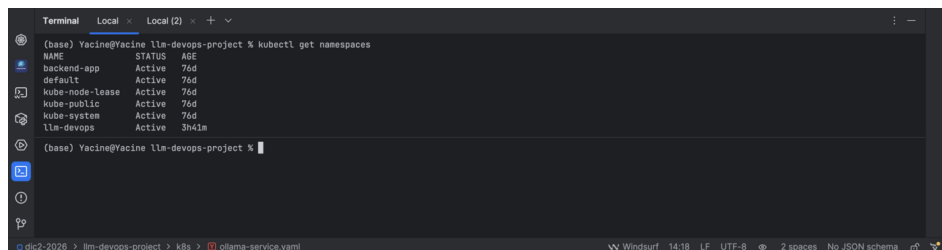
Le service `open-webui` sert l'interface web et pointe vers Ollama via la variable `OLLAMA_BASE_URL`. Les deux services communiquent sur un réseau bridge commun, avec un `healthcheck` Docker natif et une condition `depends_on: service_healthy` qui garantit qu'Open WebUI ne démarre qu'une fois Ollama réellement opérationnel.

Une fois cette base validée localement, la migration vers les briques natives de Kubernetes a pu commencer.

3 Migration vers Kubernetes

3.1 Isoler le projet dans un namespace

Toutes les ressources du projet sont regroupées dans le namespace `llm-devops`, séparé du reste du cluster (`kube-system`, etc.). Cette isolation évite les collisions de noms et permet de supprimer l'ensemble du déploiement d'un coup, sans risque pour le reste du cluster.

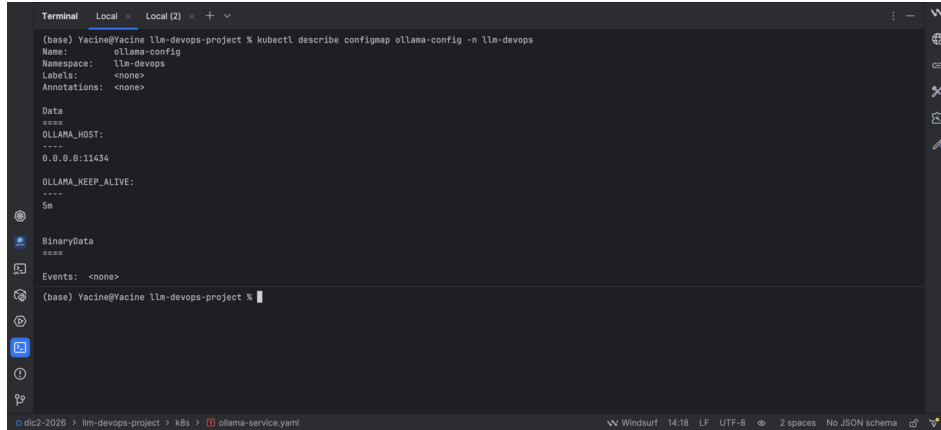


```
(base) Yacine@Yacine: llm-devops-project % kubectl get namespaces
NAME                STATUS   AGE
backend-app         Active   76d
default              Active   76d
kube-node-lease      Active   76d
kube-public          Active   76d
kube-system          Active   76d
llm-devops           Active   3h41m
(base) Yacine@Yacine: llm-devops-project %
```

FIGURE 1 – Le namespace `llm-devops`, actif

3.2 Externaliser la configuration

Les variables d’environnement ne sont pas codées en dur dans les Deployments : elles sont portées par des ConfigMaps, une pour Ollama (`OLLAMA_HOST`, `OLLAMA_KEEP_ALIVE`) et une pour Open WebUI (`OLLAMA_BASE_URL`), injectées via `envFrom`. La configuration peut ainsi changer sans reconstruire une image ni modifier le Deployment.



```
(base) Yacine@Yacine llm-devops-project % kubectl describe configmap ollama-config -n llm-devops
Name:         ollama-config
Namespace:    llm-devops
Labels:       <none>
Annotations:  <none>

Data
====
OLLAMA_HOST:
---
0.0.0.0:11434
OLLAMA_KEEP_ALIVE:
---
5s

BinaryData
====
Events: <none>
(base) Yacine@Yacine llm-devops-project %
```

FIGURE 2 – La ConfigMap d’Ollama

3.3 Persister les données

Un PersistentVolumeClaim Kubernetes est un espace de stockage totalement indépendant du volume Docker utilisé par Compose ; il n’existe aucune migration automatique entre les deux. Au premier déploiement Kubernetes, ni les modèles Ollama ni le compte Open WebUI n’étaient donc présents, ce qui a nécessité un nouveau téléchargement des modèles (détaillé en section 5). Un PVC de 15 Gi a été alloué à Ollama et un de 5 Gi à Open WebUI.



```
(base) Yacine@Yacine llm-devops-project % kubectl get pvc -n llm-devops
NAME          STATUS  VOLUME                                     CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
ollama-pvc    Bound  pvc-bd998096-f026-4a4f-95e3-b893c69c7ec7  15Gi      RWO           standard      <unset>              51m
(base) Yacine@Yacine llm-devops-project %
```

FIGURE 3 – Les deux PersistentVolumeClaim, à l’état Bound

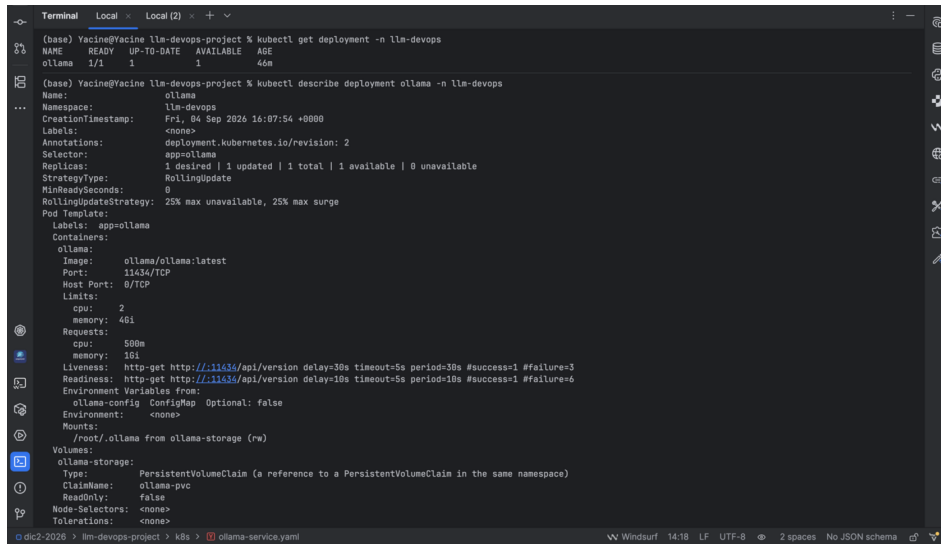


```
(base) Yacine@Yacine llm-devops-project % kubectl get pvc -n llm-devops
NAME          STATUS  VOLUME                                     CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
ollama-pvc    Bound  pvc-bd998096-f026-4a4f-95e3-b893c69c7ec7  15Gi      RWO           standard      <unset>              45h
open-webui-pvc Bound  pvc-3a4388eb-53f1-49e4-9553-0d5b58a3f30c  5Gi       RWO           standard      <unset>              17h
(base) Yacine@Yacine llm-devops-project %
```

FIGURE 4 – Détail des PVC associés à chaque service

3.4 Deployments et probes de santé

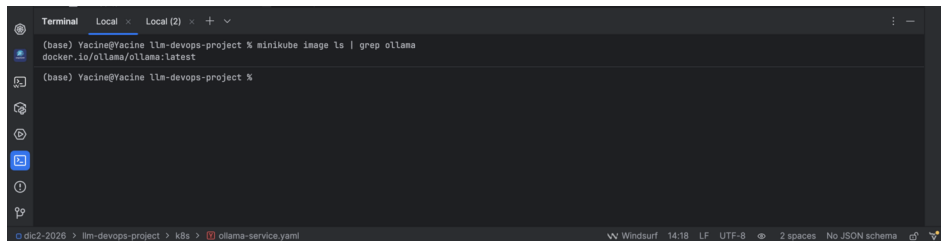
Chaque service dispose de son propre Deployment, avec des **requests/limits** de CPU et mémoire pour éviter qu'un service ne monopolise les ressources du nœud. Trois probes complètent le **healthcheck** unique de Compose : une **startupProbe**, qui laisse au conteneur jusqu'à 150 secondes pour démarrer avant que les probes suivantes ne prennent le relais (utile pour Ollama, dont le démarrage peut être lent) puis une **readinessProbe**, qui retire simplement le Pod du Service tant qu'il n'est pas prêt sans le redémarrer, et une **livenessProbe**, qui redémarre le conteneur s'il cesse de répondre.



```
(base) Yacine@Yacine llama-devops-project % kubectl get deployment -n llama-devops
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
ollama    1/1     1            1           46s

(base) Yacine@Yacine llama-devops-project % kubectl describe deployment ollama -n llama-devops
Name: ollama
Namespace: llama-devops
CreationTimestamp: Fri, 04 Sep 2026 16:07:54 +0000
Labels: <none>
Annotations: deployment.kubernetes.io/revision: 2
Selector: app=ollama
Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType: RollingUpdate
RollingUpdateStrategy: 25% max unavailable, 25% max surge
Pod Template:
  Labels: app=ollama
  Containers:
    ollama:
      Image: ollama/ollama:latest
      Port: 11434/TCP
      Host Port: 0/TCP
      Limits:
        cpu: 2
        memory: 4Gi
      Requests:
        cpu: 500m
        memory: 1Gi
      Liveness: http-get http://11434/api/version delay=30s timeout=5s period=30s #success=1 #failure=3
      Readiness: http-get http://11434/api/version delay=10s timeout=5s period=10s #success=1 #failure=6
      Environment Variables from:
        ollama-config ConfigMap Optional: false
      Environment: <none>
      Mounts:
        /root/.ollama from ollama-storage (rw)
  Volumes:
    ollama-storage:
      Type: PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
      ClaimName: ollama-pvc
      ReadOnly: false
      Node-Selectors: <none>
      Tolerations: <none>
```

FIGURE 5 – Le Deployment d'Ollama appliqué, avec ses probes et ses ressources



```
(base) Yacine@Yacine llama-devops-project % minikube image ls | grep ollama
docker.io/ollama/ollama:latest

(base) Yacine@Yacine llama-devops-project %
```

FIGURE 6 – L'image ollama/ollama:latest déployée sur le Pod

3.5 Exposer les services

Un Service est créé pour chaque Pod, fournissant un nom DNS stable (**ollama**, **open-webui**) qui ne change pas même si l'IP du Pod change. Open WebUI ne contacte donc jamais directement l'IP éphémère d'un Pod Ollama : la variable **OLLAMA_BASE_URL** pointe vers **http://ollama:11434**, résolu automatiquement par le DNS interne du cluster (CoreDNS).



```
(base) Yacine@Yacine llama-devops-project % kubectl get service -n llama-devops
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
ollama    ClusterIP  10.110.88.147    <none>           11434/TCP    2m5s

(base) Yacine@Yacine llama-devops-project % kubectl get endpoints -n llama-devops
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME      ENDPOINTS      AGE
ollama    10.244.0.29:11434    2m11s

(base) Yacine@Yacine llama-devops-project %
```

FIGURE 7 – Les Services ollama (ClusterIP) et open-webui (NodePort)

```
Terminal Local x + v
(base) Yacine@Yacine llm-devops-project % kubectl get pods -n llm-devops
NAME                                READY   STATUS    RESTARTS   AGE
ollama-5d6f656bd-mfw9g             1/1     Running   0           19m
(base) Yacine@Yacine llm-devops-project %
```

FIGURE 8 – Le Pod Ollama en Running, prêt (1/1)

```
Terminal Local x + v
(base) Yacine@Yacine llm-devops-project % kubectl get all -n llm-devops
NAME                                READY   STATUS    RESTARTS   AGE
pod/ollama-5d6f656bd-mfw9g          1/1     Running   0           43m
NAME                                TYPE     CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
service/ollama                      ClusterIP  10.110.88.147 <none>        11434/TCP   10m
NAME                                READY   UP-TO-DATE   AVAILABLE     AGE
deployment.apps/ollama              1/1     1             1             73m
NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/ollama-5d6f656bd    1         1         1       43m
replicaset.apps/ollama-7bfff9b9d     0         0         0       73m
(base) Yacine@Yacine llm-devops-project %
```

FIGURE 9 – Vue d'ensemble de tous les Pods du namespace

```
Terminal Local x + v
(base) Yacine@Yacine llm-devops-project % kubectl get pods -n llm-devops -w
NAME                                READY   STATUS    RESTARTS   AGE
ollama-65fd7d5d68-ts964            1/1     Running   0           43s
open-webui-5c6b9bb65f-4dvwq         1/1     Running   2 (45m ago)  23h
open-webui-86f7cfd6-qvkgc           0/1     Running   0           23s
open-webui-86f7cfd6-qvkgc           0/1     Running   0           75s
open-webui-86f7cfd6-qvkgc           1/1     Running   0           75s
open-webui-5c6b9bb65f-4dvwq         1/1     Terminating 2 (46m ago)  23h
open-webui-5c6b9bb65f-4dvwq         1/1     Terminating 2 (46m ago)  23h
open-webui-5c6b9bb65f-4dvwq         0/1     Completed   2 (46m ago)  23h
open-webui-5c6b9bb65f-4dvwq         0/1     Completed   2 (46m ago)  23h
open-webui-5c6b9bb65f-4dvwq         0/1     Completed   2 (46m ago)  23h
open-webui-5c6b9bb65f-4dvwq         0/1     Completed   2 (46m ago)  23h
open-webui-86f7cfd6-qvkgc           1/1     Running   0           25m
ollama-65fd7d5d68-ts964            1/1     Running   0           26m
open-webui-86f7cfd6-qvkgc           1/1     Running   0           25m
k8s > open-webui-deployment.yaml
```

FIGURE 10 – Suivi en direct d'un déploiement, du ContainerCreating au Running

```
Project dic2-2026 ~/Desktop/Documents_dic2/devOps
  dic2-2026
  > github
  > idea
  > clcd
  > Cloud
  > Docker
  > Kubernetes
  > llm-devops-project
    > k8s
      > configmap.yaml
      > namespace.yaml
      > ollama-deployment.yaml
      > ollama-pvc.yaml
      > ollama-service.yaml
      > .env
      > docker-compose.yaml
      > Dockerfile
      > README.md

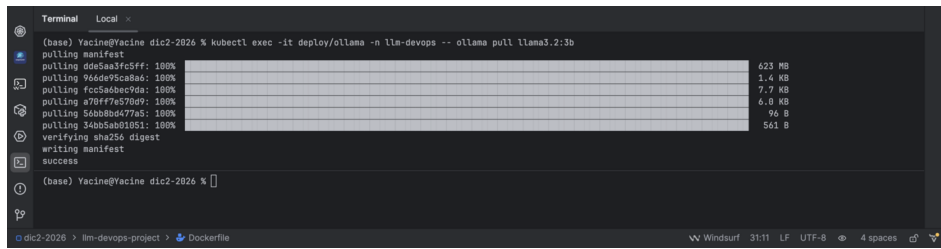
M+ README.md ollama-deployment.yaml ollama-service.yaml docker-compose.yaml
1 apiVersion: v1
2 kind: Service
3 metadata:
4   name: ollama
5   namespace: llm-devops
6 spec:
7   selector:
8     app: ollama
9   ports:
10    - name: http
11      protocol: TCP
12      port: 11434
13      targetPort: 11434
14    type: ClusterIP

Terminal Local x + v
(base) Yacine@Yacine llm-devops-project % kubectl get nodes
NAME     STATUS   ROLES    AGE   VERSION
minikube Ready    control-plane 76d   v1.35.1
(base) Yacine@Yacine llm-devops-project %
```

FIGURE 11 – Vérification générale de l'état du cluster Minikube

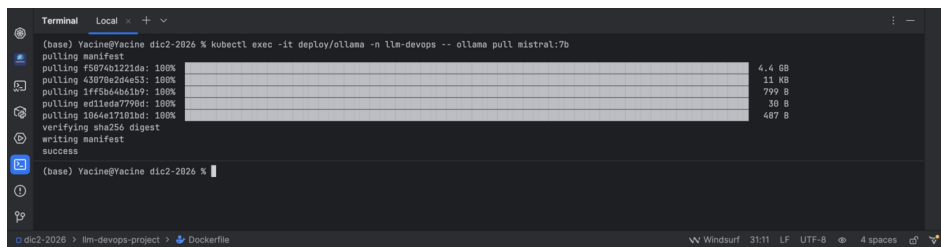
3.6 Récupérer les modèles

Comme le PVC est neuf au premier déploiement, les trois modèles ont été téléchargés directement dans le Pod.



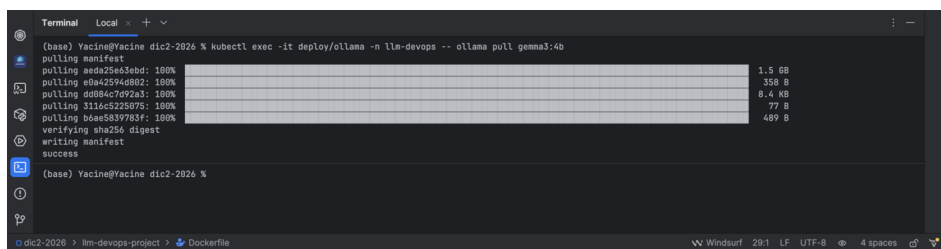
```
(base) Yacine@Yacine dic2-2026 % kubectl exec -it deploy/ollama -n llm-devops -- ollama pull llama3.2:3b
pulling manifest
pulling dde5aa3fc5ff: 100%
pulling 96ade95ca8da: 100%
pulling fcc5a4bec9da: 100%
pulling a70ff7e578d9: 100%
pulling 56bb8d477a5: 100%
pulling 34ab5a01851: 100%
verifying sha256 digest
writing manifest
success
(base) Yacine@Yacine dic2-2026 %
```

FIGURE 12 – Téléchargement de llama3.2:3b



```
(base) Yacine@Yacine dic2-2026 % kubectl exec -it deploy/ollama -n llm-devops -- ollama pull mistral:7b
pulling manifest
pulling f507da1221da: 100%
pulling 43078e2de53: 100%
pulling 1ff5b64b41b9: 100%
pulling ed18e47790a: 100%
pulling 1944e1711bd: 100%
verifying sha256 digest
writing manifest
success
(base) Yacine@Yacine dic2-2026 %
```

FIGURE 13 – Téléchargement de mistral:7b

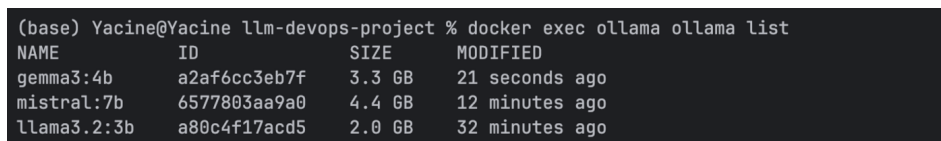


```
(base) Yacine@Yacine dic2-2026 % kubectl exec -it deploy/ollama -n llm-devops -- ollama pull gemma3:4b
pulling manifest
pulling aeda25e63ebd: 100%
pulling e8a42594d802: 100%
pulling d0b84c792a3: 100%
pulling 3116c5225075: 100%
pulling b6ae5839783f: 100%
verifying sha256 digest
writing manifest
success
(base) Yacine@Yacine dic2-2026 %
```

FIGURE 14 – Téléchargement de gemma3:4b

3.7 Tester les modèles

Une fois les trois modèles disponibles, chacun a été testé individuellement depuis l’interface Open WebUI pour confirmer qu’il générerait bien une réponse.



```
(base) Yacine@Yacine llm-devops-project % docker exec ollama ollama list
NAME          ID          SIZE      MODIFIED
gemma3:4b     a2af6cc3eb7f 3.3 GB    21 seconds ago
mistral:7b    6577803aa9a0 4.4 GB    12 minutes ago
llama3.2:3b   a80c4f17acd5 2.0 GB    32 minutes ago
```

FIGURE 15 – Les trois modèles disponibles dans le sélecteur d’Open WebUI

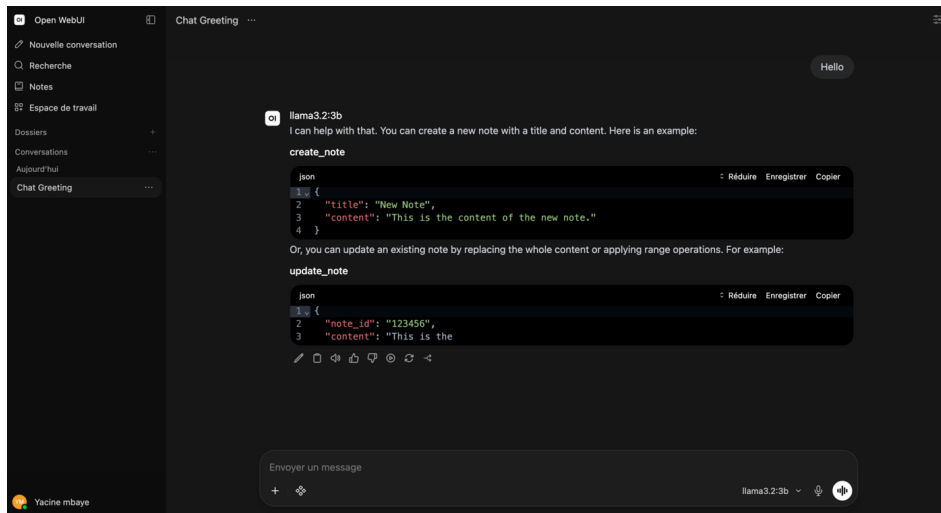


FIGURE 16 – Réponse générée par Llama 3.2

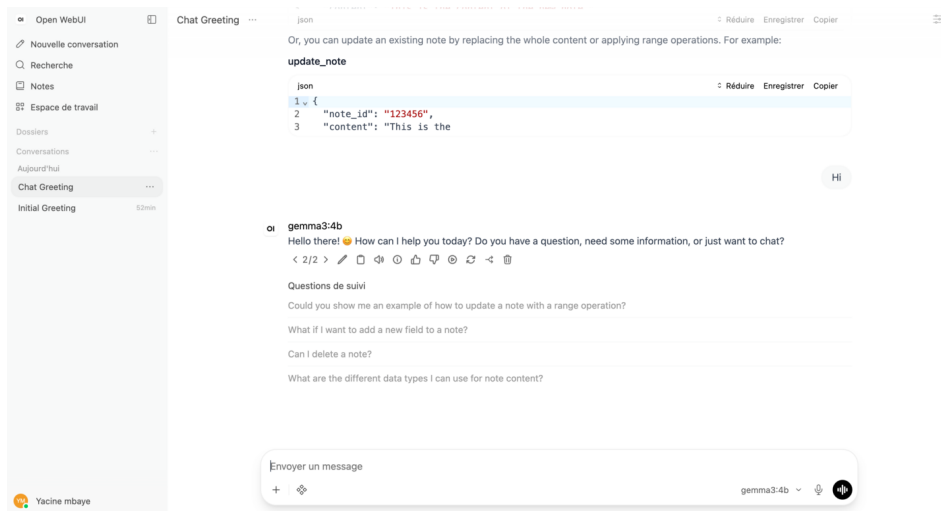


FIGURE 17 – Réponse générée par Gemma

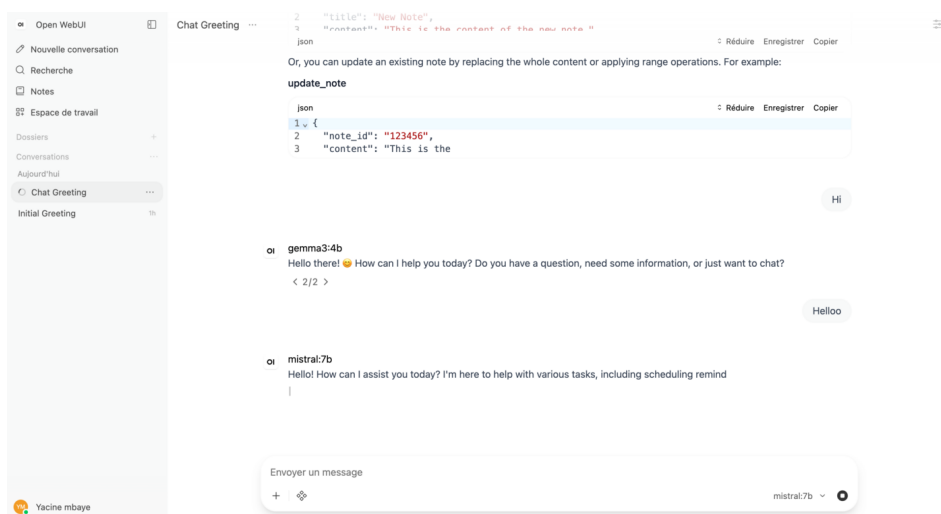
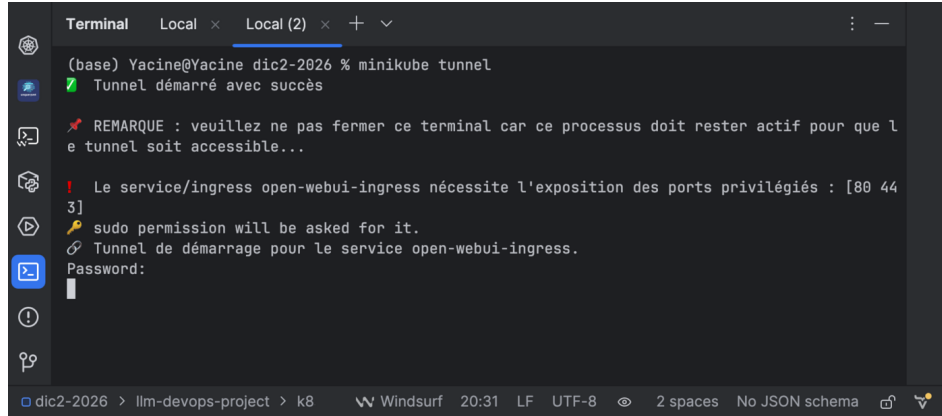


FIGURE 18 – Comparaison rapide entre Gemma et Mistral sur le même prompt

3.8 Accéder à l'application via un nom de domaine

Plutôt qu'un accès par port (NodePort), un Ingress expose Open WebUI sous le nom d'hôte `llm-devops.local`, via le contrôleur Nginx fourni par l'addon Minikube. Sur macOS avec le driver Docker, `minikube tunnel` doit rester actif en parallèle pour que ce nom d'hôte, déclaré dans `/etc/hosts`, pointe correctement vers le cluster.



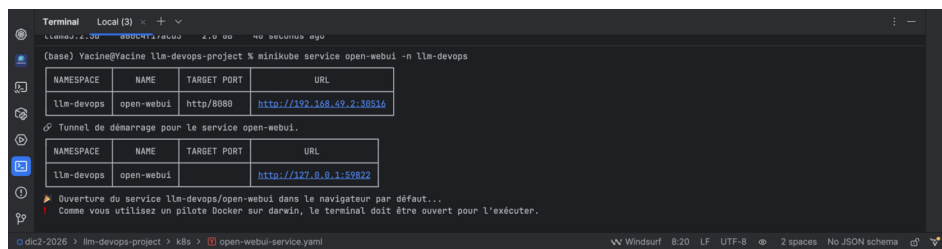
```
Terminal Local (2) x + v
(base) Yacine@Yacine dic2-2026 % minikube tunnel
✓ Tunnel démarré avec succès

REMARQUE : veuillez ne pas fermer ce terminal car ce processus doit rester actif pour que l
e tunnel soit accessible...

! Le service/ingress open-webui-ingress nécessite l'exposition des ports privilégiés : [80 44
3]
sudo permission will be asked for it.
Tunnel de démarrage pour le service open-webui-ingress.
Password:

```

FIGURE 19 – Application du manifeste Ingress



```
Terminal Local (3) x + v
(base) Yacine@Yacine llm-devops-project % minikube service open-webui -n llm-devops

NAMESPACE  NAME      TARGET PORT  URL
llm-devops  open-webui http/8080     http://192.168.49.2:38516

Tunnel de démarrage pour le service open-webui.

NAMESPACE  NAME      TARGET PORT  URL
llm-devops  open-webui http/8080     http://127.0.0.1:59822

Ouverture du service llm-devops/open-webui dans le navigateur par défaut...
! Comme vous utilisez un pilote Docker sur darwin, le terminal doit être ouvert pour l'exécuter.

```

FIGURE 20 – `minikube tunnel` actif, nécessaire sur macOS

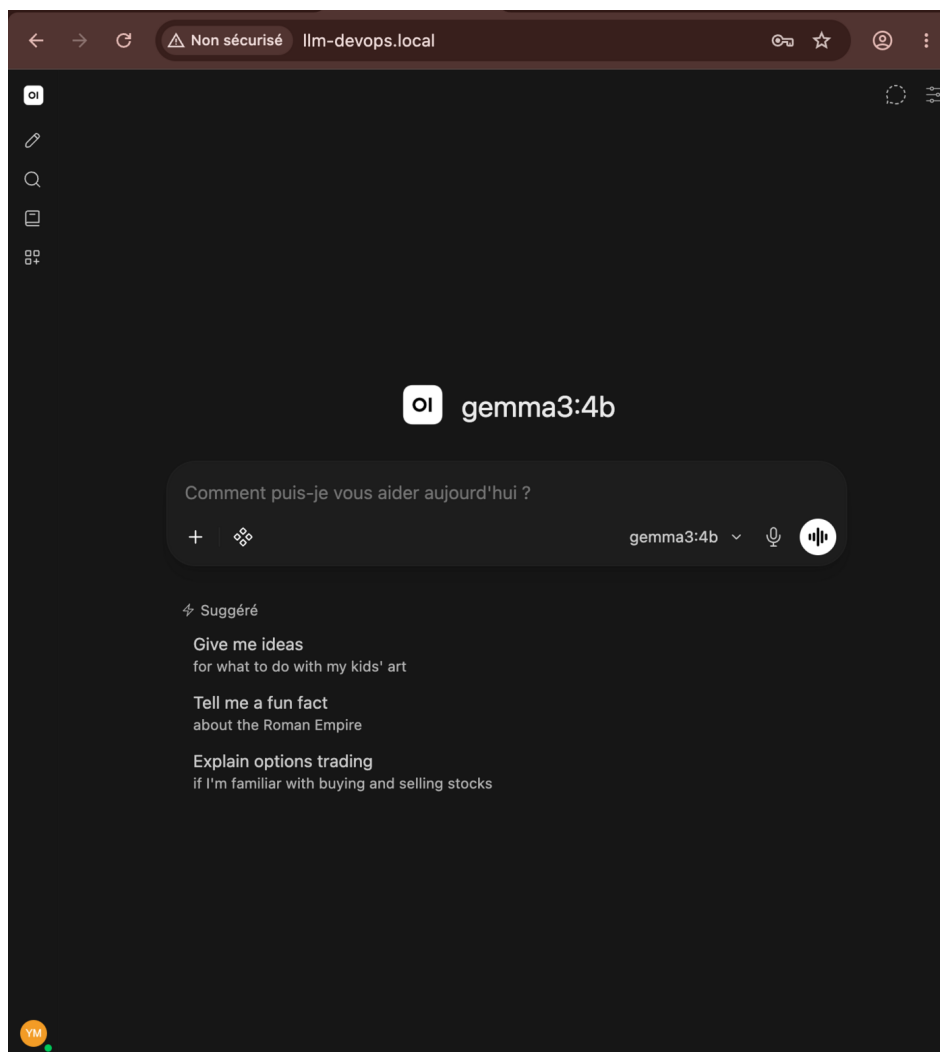


FIGURE 21 – Open WebUI accessible via `llm-devops.local`, sans préciser de port

4 Docker Compose face à Kubernetes

La migration met en évidence plusieurs différences structurelles entre les deux plateformes.

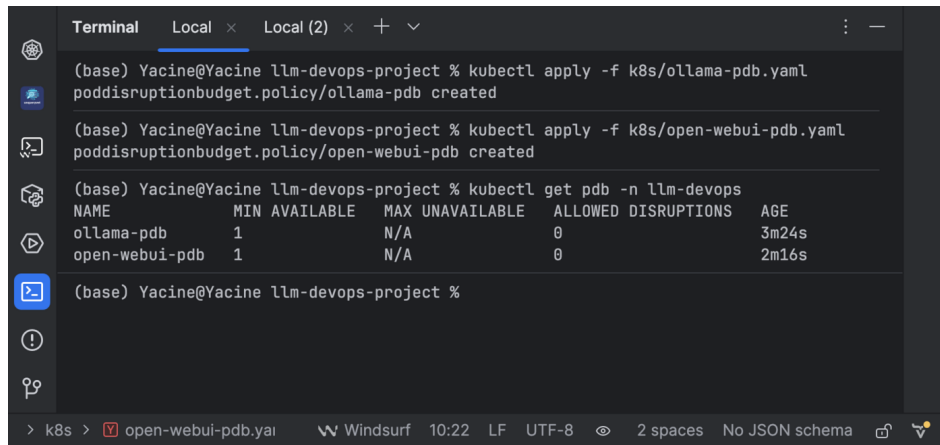
Aspect	Docker Compose	Kubernetes
Santé des services	<code>healthcheck</code> unique par conteneur	Trois probes distinctes (<code>startup</code> / <code>readiness</code> / <code>liveness</code>)
Réseau	Réseau bridge partagé, résolution par nom de service	Services + DNS interne (CoreDNS), indépendant de l'IP des Pods
Stockage	Volumes nommés, locaux au moteur Docker	<code>PersistentVolumeClaim</code> , provisionné indépendamment du cycle de vie du Pod
Auto-réparation	Redémarrage du conteneur seul	Recréation complète du Pod par le <code>Deployment</code> , avec vérification de santé
Mise à l'échelle	Manuelle (<code>docker compose scale</code>)	Déclarative (<code>replicas</code>), combinable avec un HPA

Le stockage est le point le plus structurant : un volume Docker et un PVC Kubernetes sont deux systèmes complètement séparés, ce qui explique la perte de données décrite ci-dessous.

5 Travaux bonus réalisés

5.1 PodDisruptionBudget

Un PDB protège les Pods contre les coupures volontaires (maintenance planifiée, `kubectl drain`). Avec une seule réplique par service, `minAvailable: 1` revient à interdire toute éviction volontaire ce qui illustre l'intérêt des PDB surtout en présence de plusieurs répliques.



```
(base) Yacine@Yacine llm-devops-project % kubectl apply -f k8s/ollama-pdb.yaml
poddisruptionbudget.policy/ollama-pdb created

(base) Yacine@Yacine llm-devops-project % kubectl apply -f k8s/open-webui-pdb.yaml
poddisruptionbudget.policy/open-webui-pdb created

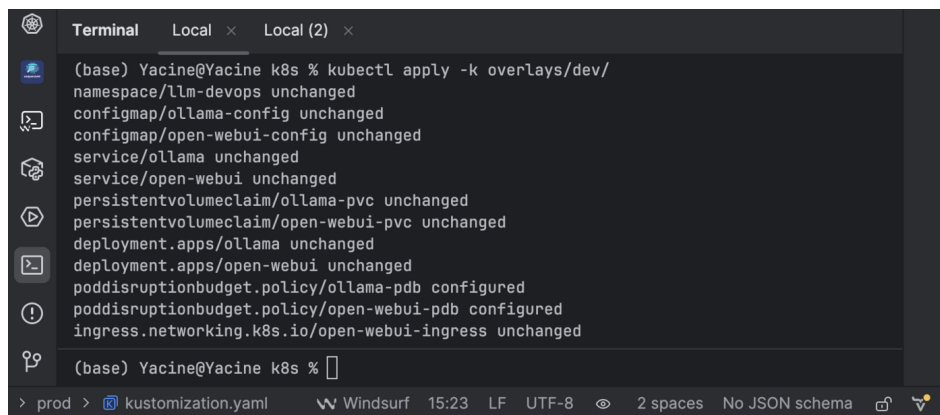
(base) Yacine@Yacine llm-devops-project % kubectl get pdb -n llm-devops
NAME           MIN AVAILABLE  MAX UNAVAILABLE  ALLOWED DISRUPTIONS  AGE
ollama-pdb     1              N/A              0                    3m24s
open-webui-pdb 1              N/A              0                    2m16s

(base) Yacine@Yacine llm-devops-project %
```

FIGURE 22 – Les deux PodDisruptionBudget, avec 0 disruption autorisée

5.2 Kustomize : séparer dev et prod

Les manifestes ont été réorganisés en une base commune (`k8s/base`) et deux overlays : `dev`, identique à la base, et `prod` qui augmente les requests/limits CPU et mémoire via des patches.



```
(base) Yacine@Yacine k8s % kubectl apply -k overlays/dev/
namespace/llm-devops unchanged
configmap/ollama-config unchanged
configmap/open-webui-config unchanged
service/ollama unchanged
service/open-webui unchanged
persistentvolumeclaim/ollama-pvc unchanged
persistentvolumeclaim/open-webui-pvc unchanged
deployment.apps/ollama unchanged
deployment.apps/open-webui unchanged
poddisruptionbudget.policy/ollama-pdb configured
poddisruptionbudget.policy/open-webui-pdb configured
ingress.networking.k8s.io/open-webui-ingress unchanged

(base) Yacine@Yacine k8s %
```

FIGURE 23 – Application de l'overlay `dev` sur le cluster réel, sans casser les Pods existants

5.3 Un pipeline CI/CD complet

Le pipeline GitHub Actions comprend deux jobs : `validate`, qui vérifie la syntaxe et le schéma des manifestes via `kubeconform` sur un runner GitHub classique, et `deploy`, qui applique réellement les changements sur le cluster Minikube local via un runner self-hosted. Ce second job est volontairement restreint aux push directs sur `main`, jamais aux Pull Requests, pour éviter qu'un tiers ne fasse exécuter du code sur la machine hôte via une PR.

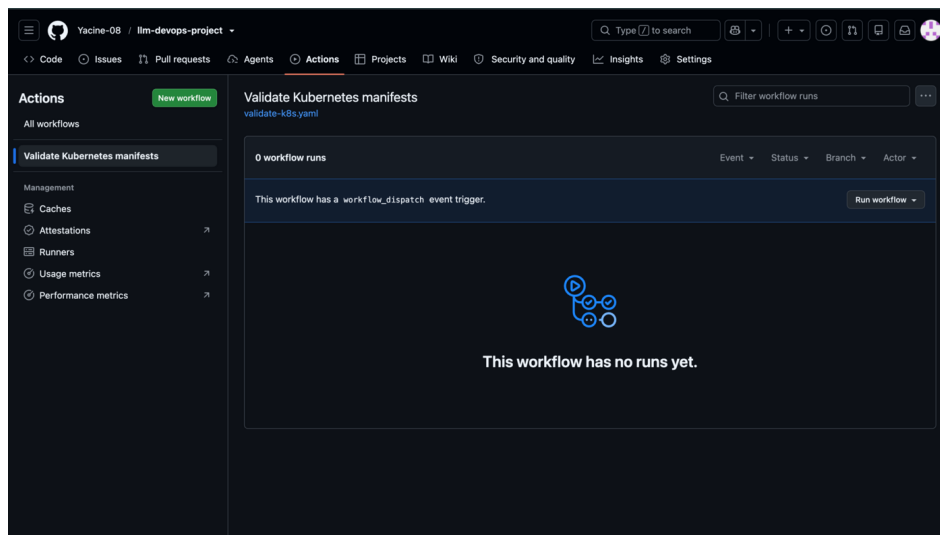


FIGURE 24 – Premier déclenchement du workflow

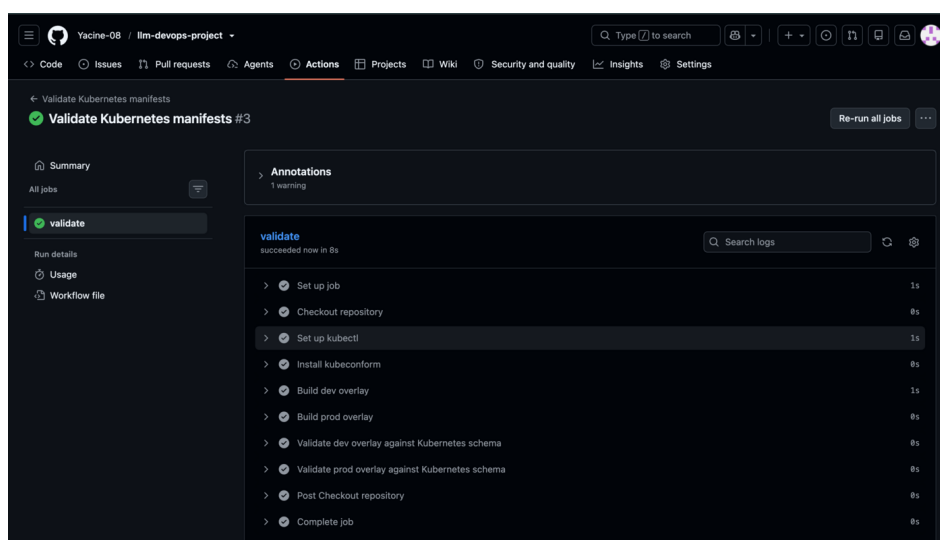


FIGURE 25 – Le job validate passé au vert

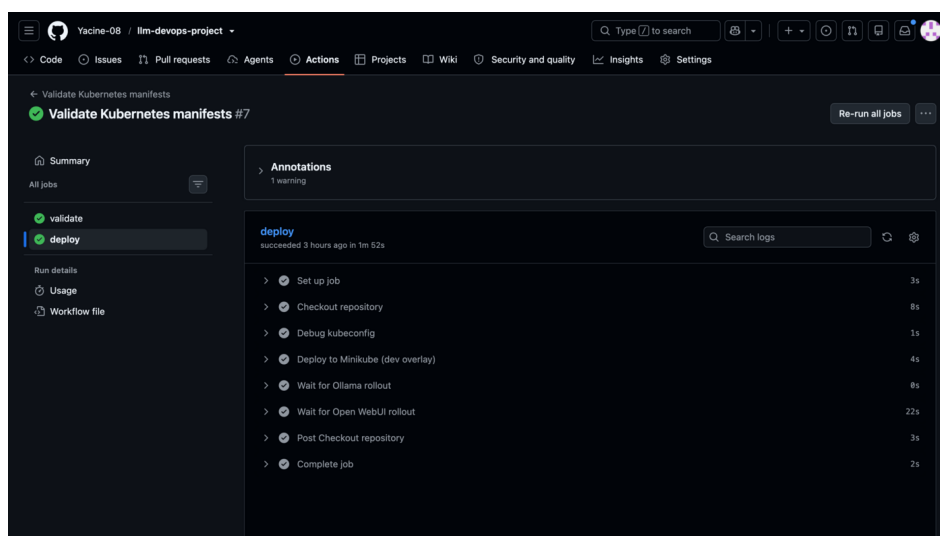


FIGURE 26 – Le job deploy, exécuté réellement sur le runner local

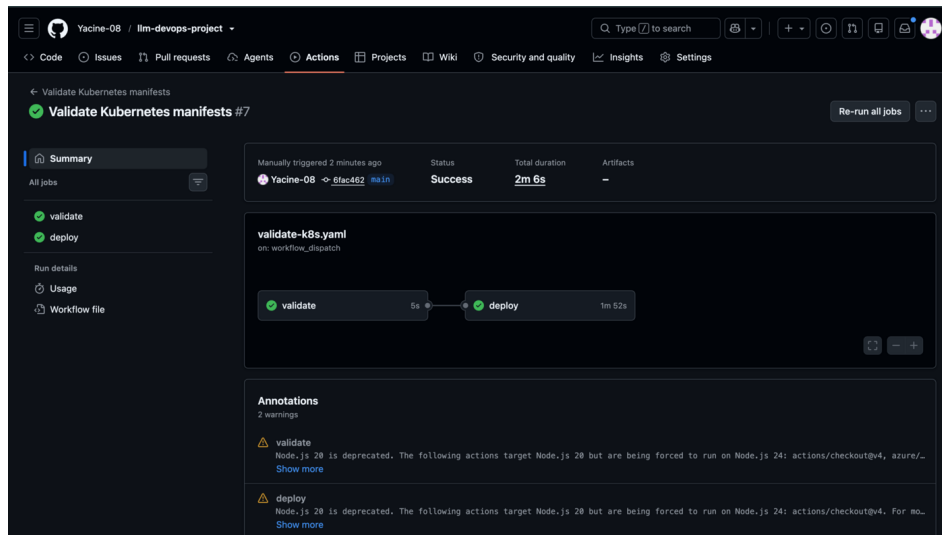


FIGURE 27 – Le pipeline complet CI et CD

5.4 Supervision avec Prometheus et Grafana

Compte tenu des ressources limitées du nœud Minikube, la stack complète `kube-prometheus-stack` (Alertmanager, node-exporter, kube-state-metrics) a été volontairement écartée au profit de l'essentiel : Prometheus interroge directement le *cAdvisor* de chaque nœud via le proxy de l'API Kubernetes et Grafana est préconfiguré pour pointer vers Prometheus dès le démarrage.

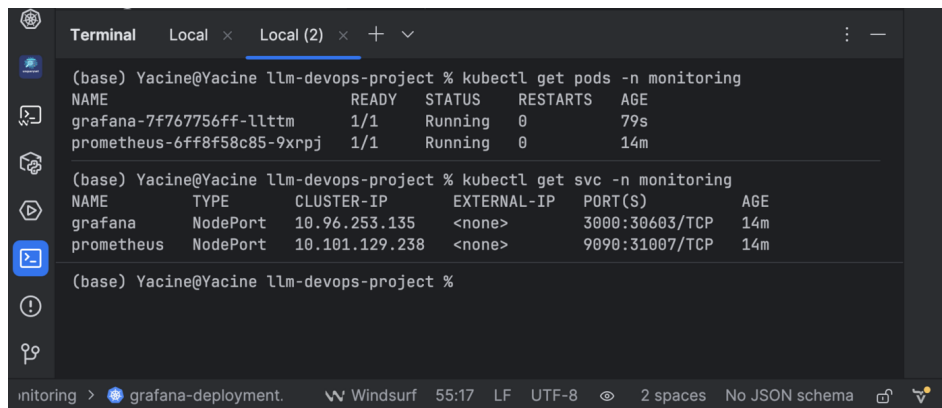


FIGURE 28 – Les Services Prometheus et Grafana créés dans le namespace monitoring

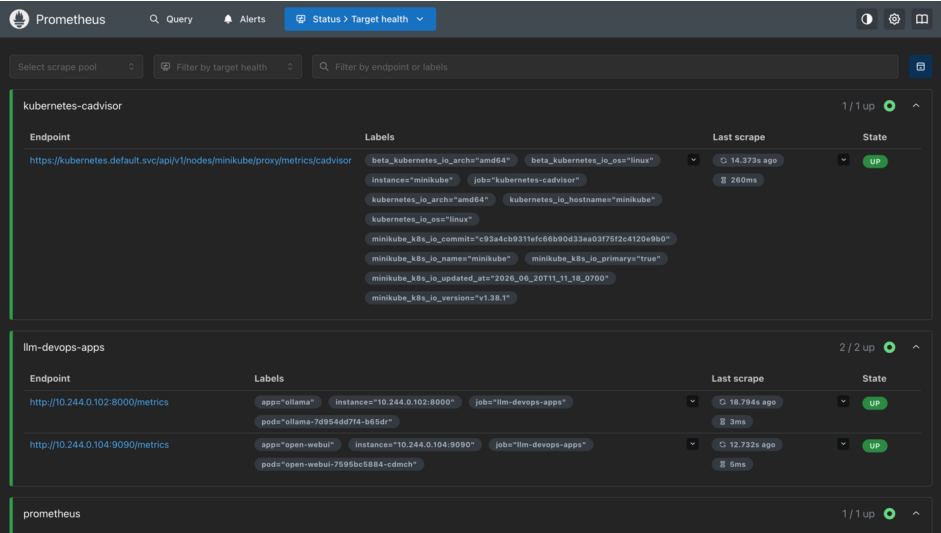


FIGURE 29 – L’interface Prometheus, avec les cibles actives

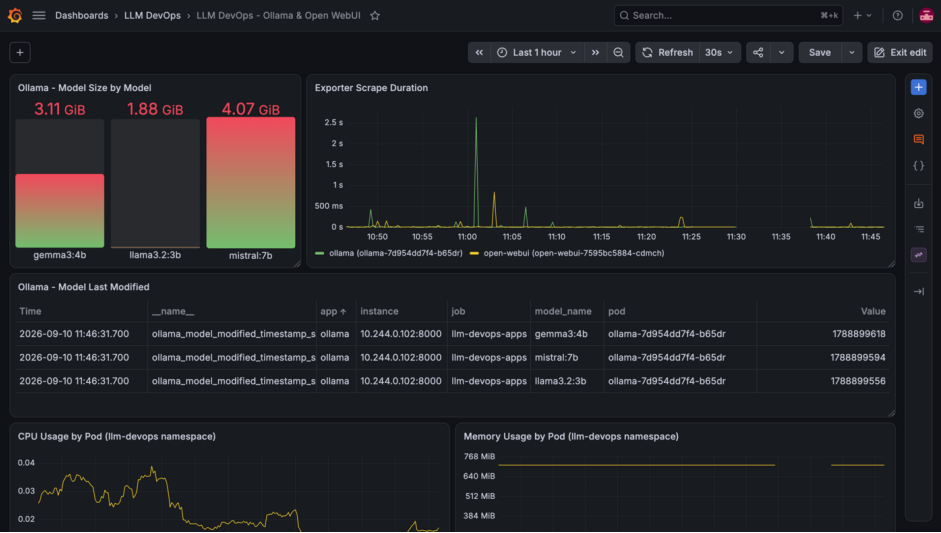


FIGURE 30 – Charge CPU/mémoire Grafana

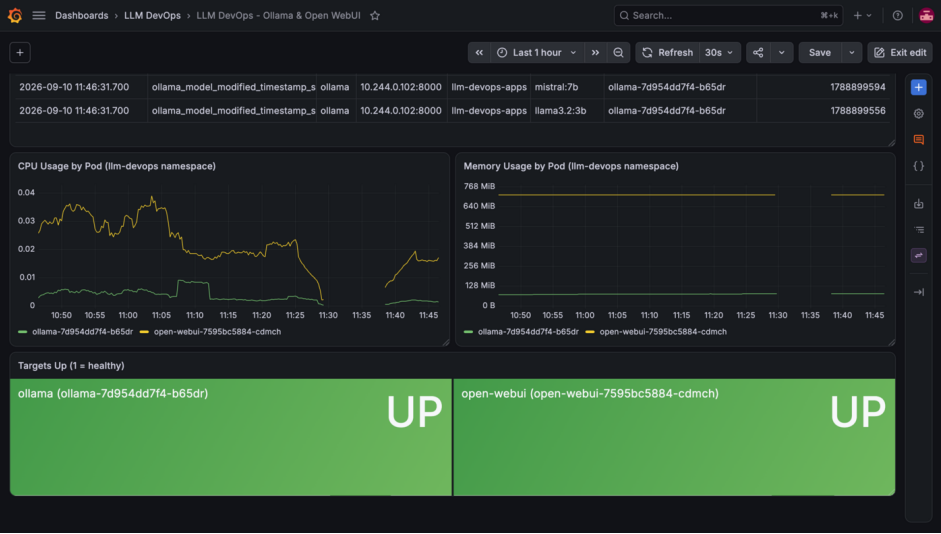


FIGURE 31 –

5.5 Autoscaling horizontal (HPA)

Un HPA a été configuré par Deployment, avec des seuils de 70 % de CPU et 80 % de mémoire, pour une plage d'une à trois répliques. Une montée en charge observée sur Open WebUI a effectivement déclenché un passage à trois répliques. Un point à surveiller : les PVC étant en `ReadWriteOnce`, il conviendrait de vérifier qu'aucune réplique supplémentaire n'entre en conflit en écrivant sur les mêmes données.

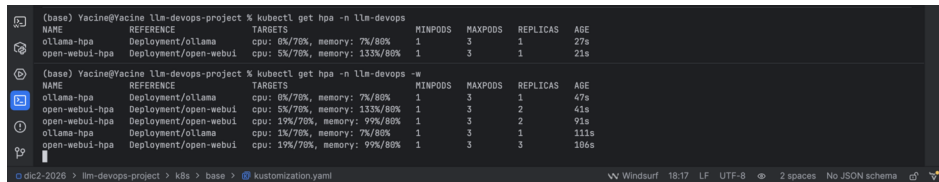


FIGURE 32 – Le HPA en action, avec la montée en répliques observée sur Open WebUI

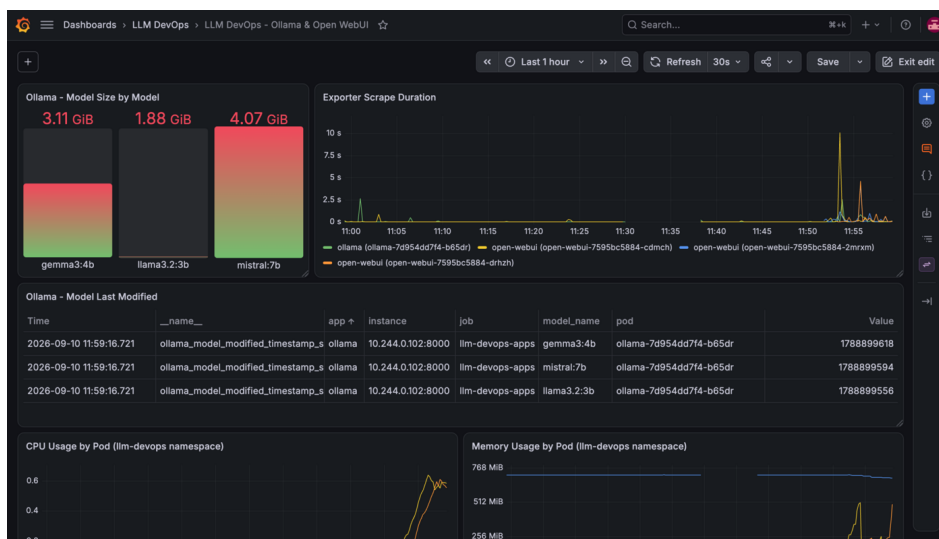


FIGURE 33 – Charge CPU/mémoire après la montée en charge

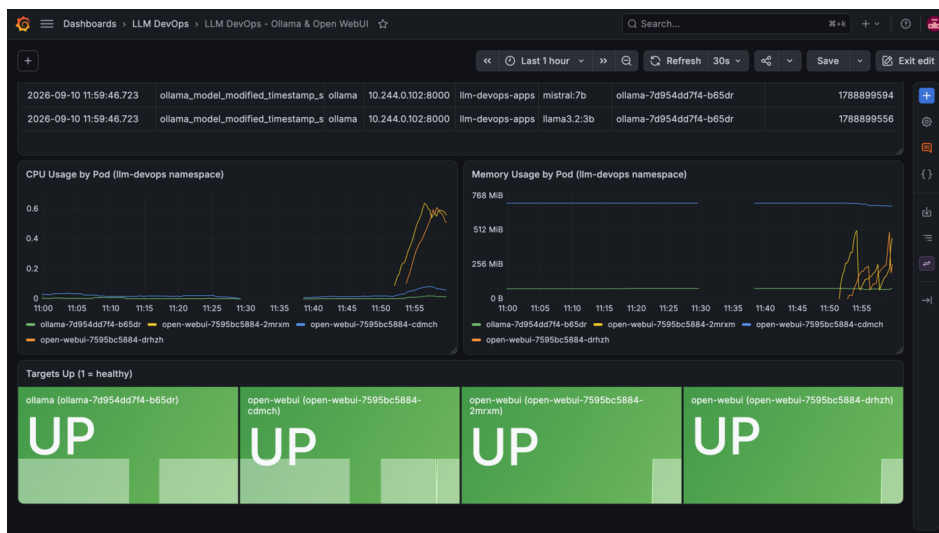


FIGURE 34 –

5.6 Restreindre le réseau avec des NetworkPolicy

Par défaut, tous les Pods d'un cluster Kubernetes peuvent se parler librement, ce qui n'est pas idéal du point de vue sécurité. Quatre politiques ont été mises en place : **default-deny-all**, qui bloque tout le trafic entrant par défaut, **allow-dns-egress**, qui préserve la résolution DNS, puis deux règles ciblées qui n'ouvrent que les flux strictement nécessaires vers Ollama et vers Open WebUI.



```
(base) Yacine@Yacine llm-devops-project % kubectl get networkpolicy -n llm-devops
NAME                                POD-SELECTOR  AGE
allow-dns-egress                    <none>        21s
default-deny-all                   <none>        27s
ollama-network-policy               app:ollama    13s
open-webui-network-policy           app:open-webui 8s

(base) Yacine@Yacine llm-devops-project % kubectl get networkpolicy -n llm-devops
NAME                                POD-SELECTOR  AGE
allow-dns-egress                    <none>        88s
default-deny-all                   <none>        94s
ollama-network-policy               app:ollama    88s
open-webui-network-policy           app:open-webui 75s

(base) Yacine@Yacine llm-devops-project %
```

FIGURE 35 – Les quatre NetworkPolicy appliquées dans le namespace

6 Conclusion

Ce projet a permis de mettre en œuvre une migration complète d'une architecture applicative de Docker Compose vers Kubernetes en couvrant l'ensemble des primitives attendues (Namespace, ConfigMap, PVC, Deployment, Service, Ingress, probes) ainsi que six des sept bonus proposés, menés jusqu'à leur validation effective.